RoboSeed: Open Experiments for Multi-Scale Vision-Language-Action Models

Reproducible Performance, Training Costs, and Closed-Loop Evaluation

RoboSeed Team

October 12, 2026

Project repository: https://github.com/PrimitiveX/AMseed

Abstract

Embodied AI is advancing rapidly, but access to its progress remains uneven. Research teams differ in computing budgets, data resources, and deployment constraints, while model releases often provide limited evidence for choosing a practical configuration or reproducing the experimental process. RoboSeed is an open experimental program designed to make robot learning more accessible, comparable, and reproducible.

The project organizes systematic training and evaluation across **model backbones and parameter scales, datasets, and action methods**, and supports rapid reproduction of emerging community techniques. Its release goals encompass performance and model weights together with the evidence needed to understand how results are obtained: training configurations, memory usage, runtime, throughput, loss convergence, rollout protocols, and failure analysis. This approach enables the community to assess capability alongside resource requirements, identify bottlenecks, and make informed choices under different budgets.

The experiments presented in this report establish an initial foundation for this program. They illustrate the trade-offs between model capacity and training cost, the importance of task-level and robustness evaluation, and the limits of interpreting optimization metrics as evidence of successful execution. As its experimental coverage expands, RoboSeed aims to provide a shared, traceable body of evidence that lowers the cost of reproduction and accelerates the adoption and improvement of embodied AI.

Keywords: vision-language-action models; backbone scale; open reproduction; training cost; closed-loop evaluation.

Contents

1 Introduction . . . **3**
- 1.1 Model Choices for Different Computing Budgets . . . 3
- 1.2 Three Directions of Contribution . . . 3
- 1.3 Scope of This Report . . . 3

2 Related Work . . . **4**

3 Model Reproduction . . . **4**
- 3.1 Reproducing Open VLA Models . . . 4
  - 3.1.1 Experimental Axes and Comparison Principles . . . 4
  - 3.1.2 Artifacts for Reproducible Releases . . . 5
  - 3.1.3 Metrics and Aggregation . . . 5
- 3.2 ABot-M0 . . . 6
  - 3.2.1 Architecture and Backbone Substitution . . . 6
  - 3.2.2 Action Learning Objective . . . 6
  - 3.2.3 Experimental Setup . . . 7
  - 3.2.4 Backbone Scale and Task Success . . . 7
  - 3.2.5 Training Memory, Runtime, Throughput, and Weights . . . 8
  - 3.2.6 Loss Convergence and Closed-Loop Monitoring . . . 10
  - 3.2.7 Rollout Concurrency and Runtime . . . 11
  - 3.2.8 Task-Level Profiles and Uneven Scale Benefits . . . 11
  - 3.2.9 LIBERO and Zero-Shot Robustness . . . 12
  - 3.2.10 RoboCasa: High-Dimensional Control and Failure Diagnosis . . . 13
  - 3.2.11 CALVIN: Continuous Language-Conditioned Task Chains . . . 14
  - 3.2.12 Interpretation and Evidence Limits . . . 15

4 Conclusion . . . **16**

5 Future Work . . . **16**

A Experimental Details and Evidence Completeness . . . **17**
- A.1 Dependencies, Interfaces, and Evaluation Settings . . . 17
- A.2 Available Evidence and Remaining Measurements . . . 18
- A.3 Reproducing the Data and Figures . . . 18

B Complete RoboTwin Task Counts . . . **19**

C CALVIN Configuration, Training, and Complete Task Counts . . . **20**

1 Introduction

1.1 Model Choices for Different Computing Budgets

VLA models provide an interface for executing robot tasks from natural-language instructions. As vision-language models (VLMs) evolve, more backbones become available, while GPU memory, training time, and inference resources remain practical constraints. A model's main released configuration may not suit every team. Whether a smaller backbone is sufficient, whether a larger one justifies its cost, and whether a new backbone's capabilities transfer to action learning require comparable experiments.

Many releases organize their evidence around a primary configuration. Final success rates alone make it difficult to reconstruct the training budget, understand convergence, or diagnose closed-loop failures. RoboSeed aims to provide a set of documented choices: substitute backbones and parameter scales within an action method, train and evaluate across datasets, and expose the resource costs and execution conditions underlying each result.

1.2 Three Directions of Contribution

Multi-scale experiments and open weights. RoboSeed organizes systematic evaluations across action methods, VLM backbones, and datasets, with performance and cost measurements for smaller and larger configurations. The aim is to help teams choose reproducible models that fit their resource budgets.

Rapid training and evaluation of emerging techniques. Modular interfaces support the integration of new backbones, action-generation mechanisms, and training strategies, followed by training, closed-loop evaluation, and open release. Compatibility, accelerated kernels, and controller interfaces are part of this workflow, so proposed benefits can be assessed under actual execution conditions.

Transparency throughout training and rollout. GPU memory, GPU-hours, throughput, loss curves, evaluator versions, task-level outcomes, concurrency, runtime, and failure videos are treated as experimental artifacts. These details help readers verify scores, estimate reproduction costs, and understand how an experiment succeeded or failed.

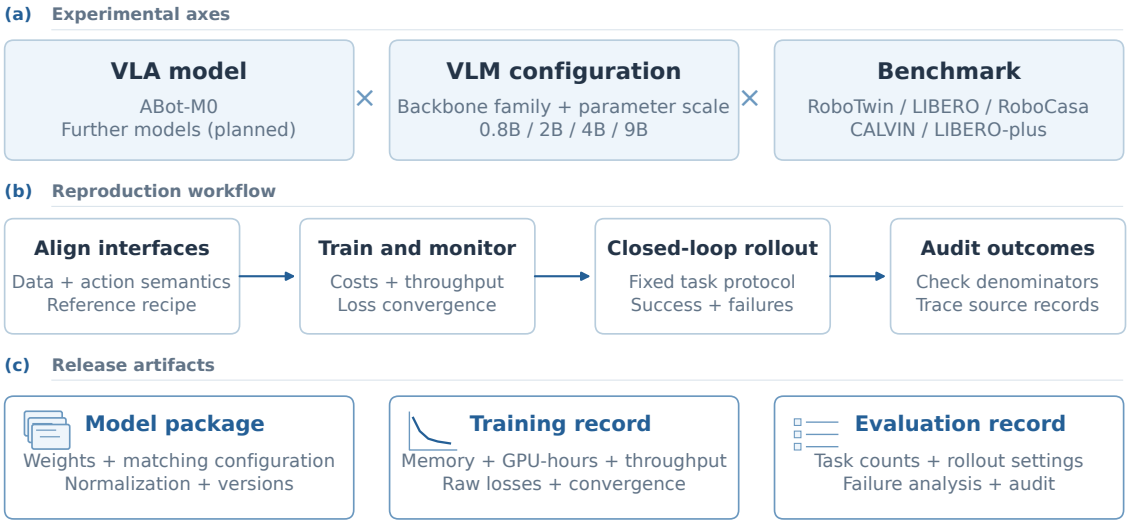


Figure 1. The RoboSeed experimental program. (a) VLA models, VLM configurations, and benchmarks define an extensible experimental matrix. (b) Reproduction proceeds through interface alignment, training, closed-loop evaluation, and outcome auditing. (c) Releases pair model packages with training and evaluation evidence; coverage and completion status are reported per experimental unit. Five RoboTwin backbones have complete evaluation, and CALVIN adds continuous task chains. Qwen3.5-2B training is complete, with evaluation in progress.

1.3 Scope of This Report

This report presents verified slices of the matrix through ABot-M0 experiments: five completed backbone-scale comparisons on RoboTwin and Qwen3.5-4B experiments on LIBERO, LIBERO-plus, RoboCasa GR1 Tabletop, and CALVIN. LIBERO, RoboCasa, and CALVIN use independently trained, dataset-specific checkpoints; LIBERO-plus evaluates the LIBERO checkpoint without additional training. The new 9B

result tests whether scaling beyond 4B improves execution under the fixed recipe, while CALVIN measures how successful skills compose across continuous task chains. Qwen3.5-2B training is documented separately until its RoboTwin evaluation is complete. RoboSeed contributes experimental reproduction, backbone-substitution studies, and process analysis. The action formulation and spatial-module design are attributed to ABot-M0.

2 Related Work

Open and compact VLA models. OpenVLA releases a 7B VLA model and fine-tuning tools for reproducing and adapting generalist policies(Kim et al., 2024). SmolVLA emphasizes affordable training, compact models, and efficient execution(Shukor et al., 2025). These projects motivate evaluating capability together with resource requirements. Previous work also examines model scale and efficiency; RoboSeed's focus is a continuously expanding experimental matrix accompanied by process evidence, performance, and weights.

Action modeling and spatial priors. ABot-M0 combines VLM semantic features, an action expert, and optional spatial modules, and introduces Action Manifold Learning (AML)(Yang et al., 2026). Its modular structure supports backbone substitution. Our experiments retain the main action architecture and training protocol while assessing different backbones under matched task settings.

Closed-loop evaluation and robustness. RoboTwin 2.0 provides bimanual manipulation tasks with domain randomization(Chen et al., 2025). LIBERO supports evaluation of spatial, object, goal, and long-horizon manipulation(Liu et al., 2023), while LIBERO-plus probes robustness across seven perturbation dimensions(LIBERO-plus, 2025). RoboCasa GR1 Tabletop includes arms, dexterous hands, and waist control(RoboCasa, 2025). These environments test different capabilities; their task distributions, action spaces, and evaluation denominators must remain explicit when interpreting results.

Continuous multi-task evaluation. CALVIN provides language-conditioned manipulation in a simulated tabletop environment(Mees et al., 2022). Its long-horizon evaluation measures the successful prefix of a chain of five instructions. This complements isolated task success: later instructions are attempted only after earlier instructions succeed, and the physical state is carried forward within a chain. We report this composition metric separately from the episode success rates of the other benchmarks.

3 Model Reproduction

3.1 Reproducing Open VLA Models

RoboSeed reproduces published vision-language-action models and organizes their implementations, training runs, evaluation outcomes, and release artifacts under a shared experimental program. Each VLA model receives a dedicated subsection that describes its architecture, adaptation choices, training settings, and verified results. As coverage grows, additional models are added as successive subsections, alongside ABot-M0 in Section 3.2. Backbone variants of the same action model remain grouped within that model's subsection.

The shared workflow begins with the source implementation and a documented reference recipe. It checks dataset and action semantics, reconstructs the required interfaces, runs training and closed-loop evaluation, and records both capability and resource costs. Reproduction of a published setting and deliberate changes to that setting are identified explicitly, so a backbone substitution or a run without the robotic pre-train stage is interpreted in its actual context. ABot-M0 is the first completed model reproduction in this report; future model sections will follow the same evidence and release conventions.

3.1.1 Experimental Axes and Comparison Principles

Let $\mathcal{B}$ denote VLM backbones and scales, $\mathcal{D}$ training datasets, and $\mathcal{M}$ action methods. The experimental space is

$$\mathcal{E} = \mathcal{B} \times \mathcal{D} \times \mathcal{M}. \tag{1}$$

Each experimental unit also records its training protocol, random seed, execution interface, and evaluator version. The three sets need not have equal cardinalities. Coverage expands as datasets, methods, and backbones are added, with the completion status of each unit recorded explicitly.

Backbone comparisons prioritize matching training data, update count, global batch size, optimizer, action-expert type, frozen modules, and evaluation protocol. Cross-attention and spatial-projection interfaces are adapted to each backbone's hidden width. **Equal update counts do not imply equal compute**:

backbone size, interface width, and kernel implementations affect per-step costs. We therefore report task performance together with observed resource consumption rather than interpreting the study as a strictly equal-FLOPs scaling law.

3.1.2 Artifacts for Reproducible Releases

A RoboSeed release is intended to contain a complete experimental unit. Table 1 describes the proposed artifact protocol; Appendix A distinguishes available evidence from missing measurements.

Table 1. RoboSeed artifact protocol. Each release should state its evidence completeness.

Artifact	Contents	Purpose
Configuration and environment	Dataset versions, backbone sources, code commits, dependencies, seeds, and effective configuration snapshots	Reconstruct execution conditions
Training process	Updates, batch size, learning rates, frozen modules, loss curves, memory, throughput, wall time, and GPU-hours	Assess budgets and convergence
Model weights	Checkpoints, matching configurations, normalization statistics, file sizes, and checksums	Keep weights and execution interfaces consistent
Rollout process	Scenes, instructions, episodes, action chunks, step limits, concurrency, runtime, retries, and exclusion rules	Verify denominators and execution settings
Results and diagnosis	Task-level counts, failure videos, anomalies, diagnostic evidence, and untested hypotheses	Support verification and improvement

3.1.3 Metrics and Aggregation

For task t, success rate is $\mathrm{SR}_t = s_t/n_t$. The task macro-average is $T^{-1}\sum_t \mathrm{SR}_t$, and the episode micro-average is $\sum_t s_t / \sum_t n_t$. Each RoboTwin scene evaluates every task for 100 episodes, so these averages coincide within a scene. The two-scene average is $(\mathrm{SR}_{\mathrm{Clean}} + \mathrm{SR}_{\mathrm{Randomized}})/2$. LIBERO and RoboCasa likewise use equal episode counts per task.

LIBERO-plus evaluates each perturbation variant once, with different numbers of variants across suites and categories. Its **overall score is the micro-average across all variants**; category scores use their actual denominators. Scores from different benchmarks are not pooled. Differences between standard LIBERO and LIBERO-plus describe performance under their respective test distributions.

For CALVIN, let $\ell_i \in \{0, \ldots, 5\}$ be the successful prefix length of sequence i, and $C_k = N^{-1}\sum_i \mathbf{1}[\ell_i \geq k]$. The primary metric is

$$\bar{\ell} = \frac{1}{N}\sum_{i=1}^{N} \ell_i = \sum_{k=1}^{5} C_k. \tag{2}$$

We also report $C_1, \ldots, C_5$ and task-specific successes divided by actual attempts. Task denominators are unequal because a failed subtask terminates its chain; these conditional rates are diagnostic and are not substituted for the sequence metric.

3.2 ABot-M0

3.2.1 Architecture and Backbone Substitution

Figure 2 shows the implemented module relationships. Multi-view images and language instructions are encoded by the VLM. A VGGT stream provides geometric features, which are projected to the backbone's hidden width and fused through cross-attention. The action expert conditions on fused features, available robot state, and noisy actions to predict an action sequence. Predictions are unnormalized using dataset statistics and passed to the controller.

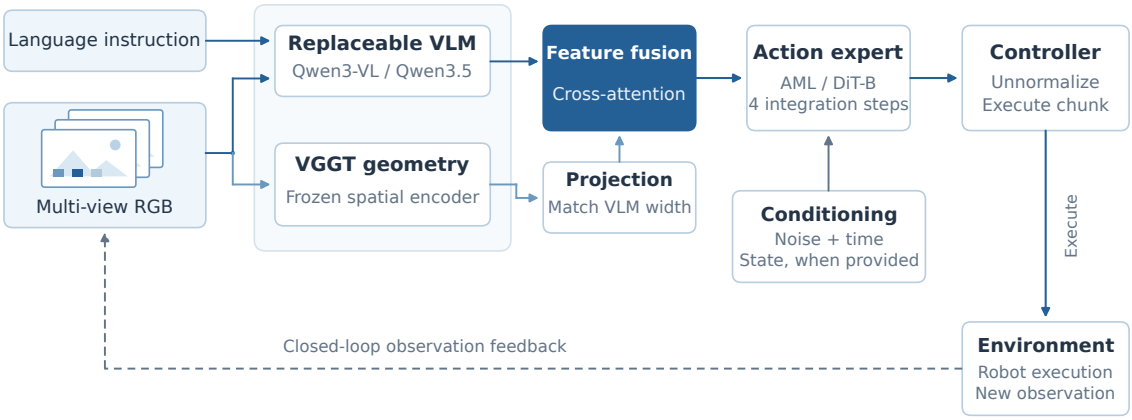


Figure 2. ABot-M0 backbone substitution and action generation, redrawn from the implementation interfaces. Feature widths are adapted when replacing the VLM; VGGT is frozen during these experiments. Dark blue arrows denote semantic and action flow, light blue arrows denote geometry flow, and the dashed connection denotes observation feedback from execution. State conditioning is used when supplied by the execution interface.

3.2.2 Action Learning Objective

For an expert action chunk A, noise ϵ, and time $\tau \in [0,1)$, the noisy input is $A^\tau = \tau A + (1-\tau)\epsilon$. The action expert predicts a clean chunk $\hat{A} = f_\theta(\phi, q, A^\tau, \tau)$, where ϕ denotes fused features and q the state condition when supplied. The implementation derives velocity fields from the predicted and target action chunks:

$$\hat{v} = \frac{\hat{A} - A^\tau}{\max(1-\tau, \delta)}, \quad v = \frac{A - A^\tau}{\max(1-\tau, \delta)}, \quad \mathcal{L} = \mathbb{E}\left[\|\hat{v} - v\|^2\right]. \tag{3}$$

Here δ ensures numerical stability. When action dimensions are padded, the loss is computed over valid dimensions. All loss curves in this report come from actual training logs. They describe optimization of the learning objective; task execution is assessed by closed-loop success.

3.2.3 Experimental Setup

Table 2. Verified experimental coverage in the current snapshot. Complete scores are derived from original evaluation artifacts.

Benchmark	Backbone	Training protocol	Evaluation coverage
RoboTwin	Qwen3.5-0.8B / 4B / 9B Qwen3-VL-2B / 4B	150k updates Final weights	50 tasks × 100 episodes per scene; 2 scenes
LIBERO	Qwen3.5-4B	40k checkpoint	4 suites; 10 tasks per suite; 50 episodes per task
LIBERO-plus	LIBERO-trained Qwen3.5-4B	Zero-shot No extra training	10,030 variants, 1 episode each
RoboCasa GR1	Qwen3.5-4B	50k checkpoint	24 tasks × 50 episodes
CALVIN	Qwen3.5-4B	34k updates Final weights	ABCD → D; 1,000 sequences of up to 5 tasks
RoboTwin extension	Qwen3.5-2B	150k updates complete	Evaluation in progress; no complete score

RoboTwin training. The five evaluated configurations and the training-only Qwen3.5-2B extension use four H200 NVL GPUs, global batch size 64 (16 per GPU, accumulation 1), and 150,000 updates. They initialize from the corresponding VLM and proceed directly to the downstream post-train stage (supervised fine-tuning, SFT), without the unified robotic pre-train stage. These scores therefore describe the reproduced training setting rather than the official pre-trained release. The training data include Clean and Randomized demonstrations; experimental records report 27,500 trajectories and 6,075,103 transitions. Image resolution is 224×224, the training seed is 42, and the spatial module is frozen. Base and VLM learning rates are 10^{-5}; the action-expert rate is 10^{-4}. AdamW uses $\beta = (0.9, 0.95)$, with 5,000 warmup updates and a cosine schedule to 5×10^{-7}. The action horizon is 50; configured state and action dimensions are both 14. The action expert is DiT-B, with four repeated time samples per training example. Dependency details appear in the appendix.

RoboTwin evaluation. Every configuration evaluates 50 tasks for 100 episodes each under both `demo_clean` and `demo_randomized`, using unseen instructions and launch seed 0. The recorded evaluator commit is `13c3c47`, with the interface and frame-sampling modifications used during these runs. Scores use binary environment success counts, without mixing in progress scores. Since training includes randomized demonstrations, Randomized scores describe performance in randomized scenes; they do not establish generalization from exclusively Clean training. Comparisons with public leaderboards require matching the evaluator version and execution protocol.

Training on other datasets. LIBERO and RoboCasa are trained independently. LIBERO uses 40k updates, global batch 64, and base/VLM/action learning rates of 2.5×10^{-5}, 3×10^{-5}, and 10^{-4}. Its action horizon is 10, configured state dimension 7, and internal action representation 14-dimensional, with the active Franka channels used for execution. The saved training script specifies `include_state=false`; its effective behavior requires checking the corresponding implementation version. RoboCasa uses 24 datasets with 1,000 demonstrations each, 50k updates, and learning rates of 10^{-5}, 10^{-5}, and 10^{-4}. Its horizon is 16, state encoding uses sine and cosine of 29 joint values (58 dimensions), and actions are 29-dimensional absolute joint targets. Both experiments freeze the spatial module.

3.2.4 Backbone Scale and Task Success

Table 3. ABot-M0 backbone comparison on RoboTwin. Rates are percentages; Average equally weights the two scenes.

Backbone	Clean count	Clean	Randomized count	Randomized	Average
Qwen3.5-0.8B	4038/5000	80.76	4025/5000	80.50	80.63
Qwen3-VL-2B	4183/5000	83.66	4195/5000	83.90	83.78
Qwen3-VL-4B	4285/5000	85.70	4267/5000	85.34	85.52
Qwen3.5-4B	4294/5000	85.88	4256/5000	85.12	85.50
Qwen3.5-9B	4199/5000	83.98	4184/5000	83.68	83.83

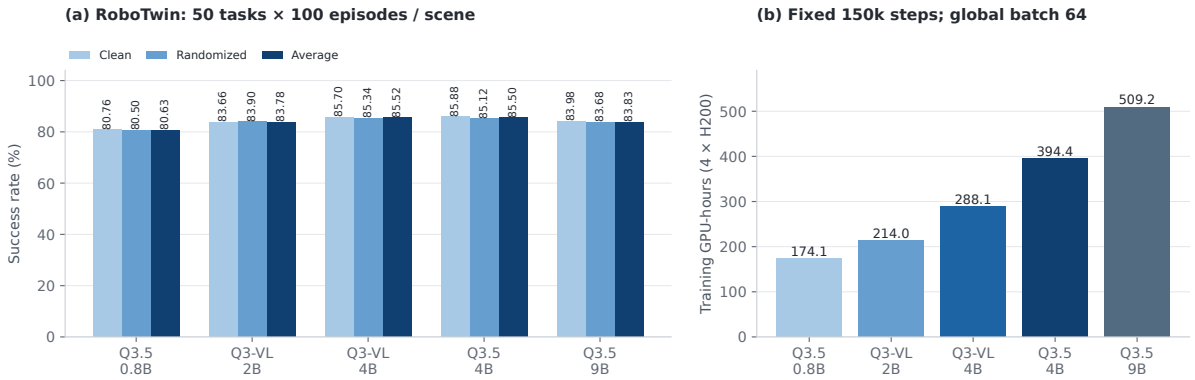


Figure 3. Task performance and training cost. Every configuration uses 150k updates and global batch 64. Bars start at zero. Training GPU-hours equal four GPUs multiplied by training wall time; they exclude evaluation and do not indicate equal-FLOPs budgets.

Smaller backbones retain substantial capability. Qwen3-VL-2B reaches 83.78%, trailing the same-family 4B score of 85.52% by 1.74 percentage points, while reducing GPU-hours from 288.1 to 214.0 (approximately 25.7%). Qwen3.5-0.8B trails its 4B counterpart by 4.87 points while reducing GPU-hours from 394.4 to 174.1 (approximately 55.8%). This evidence supports smaller backbones as useful options for constrained budgets under the tested recipe. It does not establish that these full VLA configurations can be trained on arbitrary consumer GPUs.

The two 4B backbones have similar aggregate scores. Qwen3-VL-4B and Qwen3.5-4B average 85.52% and 85.50%. Relative to Qwen3-VL, Qwen3.5 changes Clean success by $+0.18$ percentage points and Randomized success by -0.22 points. The experiment provides no clear aggregate success-rate advantage for the newer backbone. A single training seed does not establish statistical equivalence, and differences in architecture or pre-train recipes between families cannot be attributed to parameter count alone.

Scaling beyond 4B does not improve this recipe. The completed Qwen3.5-9B evaluation yields 4,199/5,000 Clean and 4,184/5,000 Randomized successes, for a two-scene average of 83.83%. This is 1.67 percentage points below Qwen3.5-4B while consuming 509.2 rather than 394.4 training GPU-hours, an increase of approximately 29.1%. Its average is only 0.05 points above Qwen3-VL-2B, despite approximately 2.38 times the training-device time. Under the tested recipe, expanding from smaller backbones to 4B improves the observed averages, but expanding further to 9B does not. With one training seed per configuration, these observations do not establish a universal optimum or identify the cause of the 9B result.

A second 2B family is being evaluated. Qwen3.5-2B has completed 150k updates and saved final weights. It requires 53.23 hours on four H200 GPUs (212.9 GPU-hours), processes approximately 50.1 nominal samples/s, and saves an 8.33 GB checkpoint. Its final logged action loss is approximately 1.44×10^{-3}. RoboTwin evaluation is still in progress at the snapshot cutoff; training completion and low loss are insufficient to assign a task-success score or rank it against Qwen3-VL-2B.

3.2.5 Training Memory, Runtime, Throughput, and Weights

Table 4. RoboTwin training process. Approximate memory readings retain the original records' GiB labels; uniformly sampled memory traces are unavailable. Throughput is recomputed from updates, batch size, and complete wall time.

Backbone	Hours	GPU-h	Samples/s	Memory / GPU (GiB) Mean	Recorded max.	Weights (GB)
Qwen3.5-0.8B	43.53	174.1	61.3	∼52.27	∼55.06	5.04
Qwen3-VL-2B	53.51	214.0	49.8	∼54.90	∼55.30	7.77
Qwen3-VL-4B	72.03	288.1	37.0	∼83.26	∼83.93	12.58
Qwen3.5-4B	98.59	394.4	27.0	∼142.50	∼143.00	13.28
Qwen3.5-9B	127.30	509.2	20.9	∼137.04	∼139.69	21.88
Qwen3.5-2B†	53.23	212.9	50.1	–	–	8.33

Training durations for the five evaluated backbones are 43:31:53, 53:30:44, 72:01:59, 98:35:29, and 127:18:06 in table order. The training-only Qwen3.5-2B run takes 53:13:34. † marks completed training with evaluation still in progress; missing memory readings are shown as dashes. Throughput is the nominal number of batch samples processed per second, not unique data items. Checkpoint sizes come from actual file byte counts and use decimal GB (10^9 bytes). Saved weights include action, spatial, and other modules and cannot be inferred from the nominal VLM size. GPU-hours measure allocated training-device time, rather than energy, utilization, or monetary cost.

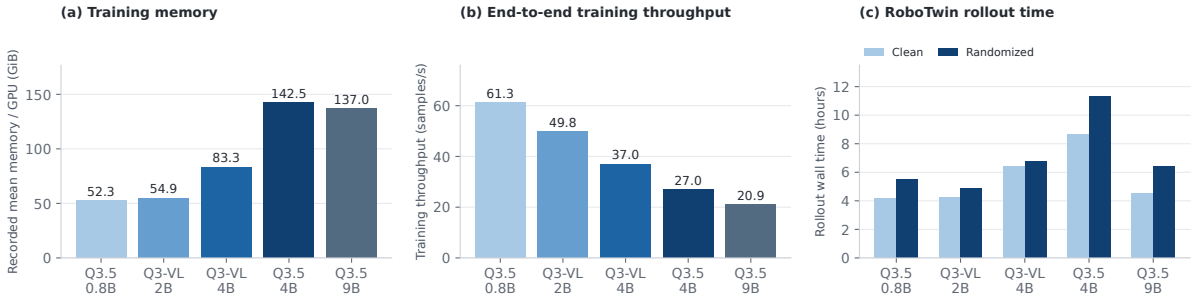


Figure 4. Training resources and rollout details. Memory bars use approximate recorded readings. The high Qwen3.5-4B reading is associated with its runtime implementation. Evaluation times retain each run's actual concurrency settings and are not measurements of single-policy inference latency.

Engineering paths matter when comparing costs. At 4B, Qwen3.5 takes approximately 36.9% more training wall time and has approximately 26.9% lower nominal throughput than Qwen3-VL. The troubleshooting records identify unavailable linear-attention acceleration components and a slower PyTorch fallback path. These measurements describe the software environment used in this experiment. The benefits of enabling the accelerated implementation require a new measurement.

Memory evidence combines approximate monitoring readings with the units stated in the original records. In particular, the 142–143 readings need reconciliation with device capacity, monitoring tools, and definitions of allocated and reserved memory before formal release. We preserve these records transparently rather than treating them as exact minimum hardware requirements obtained with a uniform profiler.

The 9B run records approximately 137.04 GiB mean and 139.69 GiB peak memory, a 21.88 GB checkpoint, and 20.9 nominal samples/s. Its recorded memory is below the 4B Qwen3.5 reading despite its larger backbone. Experiment notes report acceleration dependencies installed for later runs, but no matched before/after 4B experiment isolates their effect. Consequently, memory ordering should not be interpreted as a parameter-scaling rule or a measured kernel speedup.

3.2.6 Loss Convergence and Closed-Loop Monitoring

Figure 5 separates the six backbones so that each training trajectory can be read directly. Compare the initial decrease and continued late learning on identical logarithmic axes, then compare the late-window loss summary with the independently measured RoboTwin success rate.

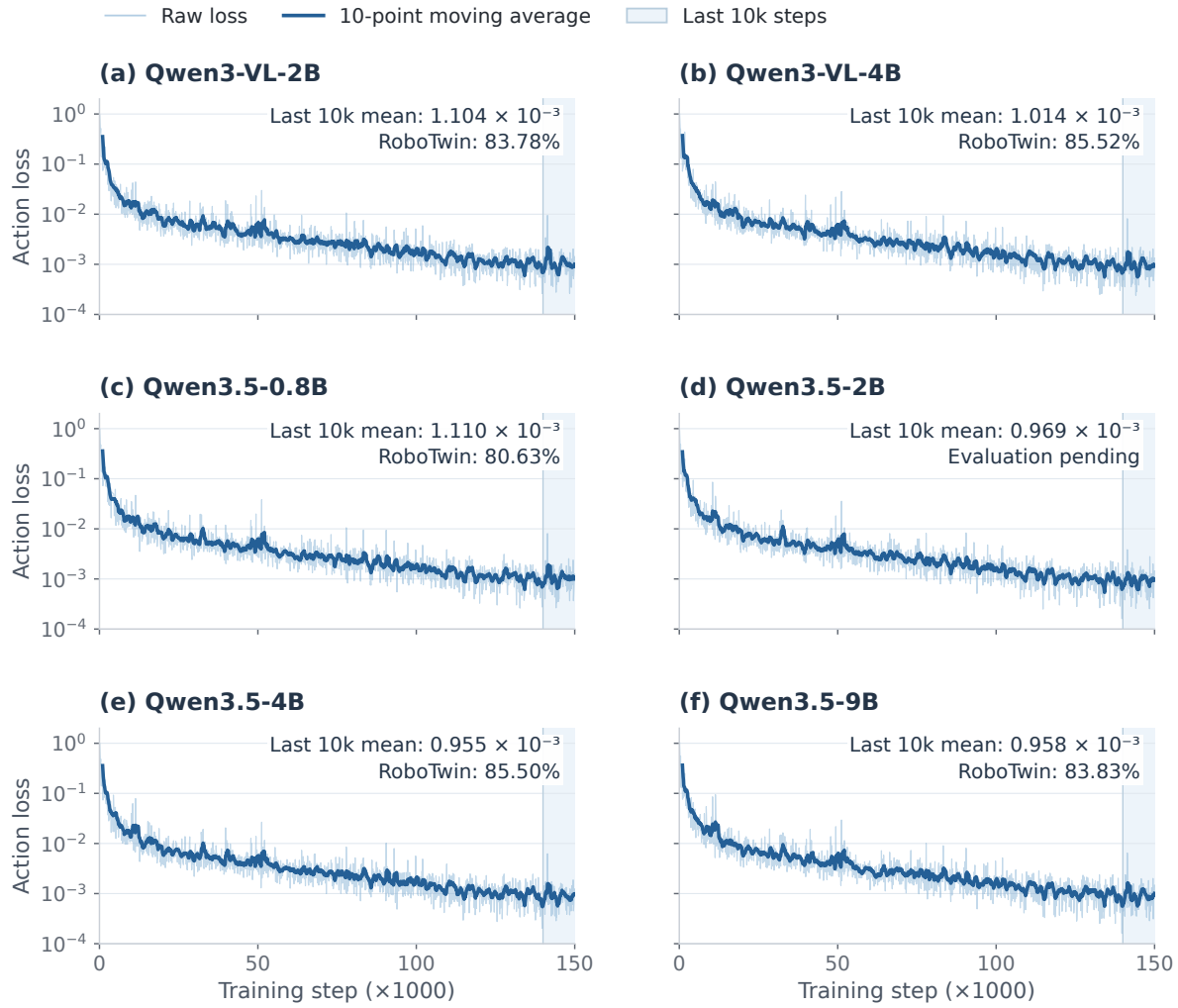


Figure 5. RoboTwin training convergence, one backbone per panel. Identical logarithmic axes show both the initial loss decrease and continued late learning over 0–150k updates, with a lower limit of 10^{-4}. Light curves retain raw observations; dark curves show a ten-point moving average. The shaded 140k–150k window defines each annotated mean: the 100 raw observations with $140{,}000 < \text{step} \leq 150{,}000$. RoboTwin rates equally weight Clean and Randomized scenes; Qwen3.5-2B evaluation is pending. Loss curves describe optimization, while closed-loop success measures task capability.

Similar tail losses do not imply similar task success. Qwen3.5-4B and Qwen3.5-9B have nearly identical last-10k mean losses (0.955×10^{-3} and 0.958×10^{-3}), yet their RoboTwin success rates are 85.50% and 83.83%. The panels show optimization progress; the rollout measurements establish task capability.

All six losses decrease and reach low values in late training. Final logged action losses range from approximately 1.23×10^{-3} to 1.47×10^{-3}. Their ordering differs from that of average task success: the 0.8B variant's lower final loss does not translate into higher success than the 4B variants. Curves help identify divergence, plateaus, and spikes; closed-loop outcomes determine whether tasks are completed.

Intermediate-checkpoint rollouts complement loss monitoring. For Qwen3.5-4B, experimental records report Randomized success of 81.4% at 110k and 85.8% at 130k, using ten episodes per task. These small evaluations screen for capability changes during training. The final checkpoint reaches 85.12% with 100 episodes per task. Differences between these evaluation sizes do not establish late-training degradation, particularly when individual ten-episode estimates are sensitive to sampling.

The 9B model finishes with action loss approximately 1.47×10^{-3}, while the new Qwen3.5-2B finishes near 1.44×10^{-3}. The Qwen3.5-2B panel in Figure 5 explicitly marks its task-success score as pending. These final single observations differ from the last-10k means annotated in the panels. Loss magnitudes from different datasets are not pooled, since action representations, valid channels, and task distributions differ.

3.2.7 Rollout Concurrency and Runtime

Table 5. Complete RoboTwin rollout details. Every configuration uses four evaluation GPUs. Clean and Randomized times are workflow wall times and may overlap.

Backbone	Max. workers/GPU	Clean (h)	Randomized (h)	Episodes
Qwen3.5-0.8B	6	4.15	5.52	10,000
Qwen3-VL-2B	6	4.26	4.88	10,000
Qwen3-VL-4B	4	6.38	6.77	10,000
Qwen3.5-4B	4	8.68	11.31	10,000
Qwen3.5-9B	4	4.51	6.38	10,000

Evaluation separates policy serving from simulation and uses queues to limit concurrency. The evaluated 0.8B and 2B configurations allow up to six workers per GPU, while both 4B configurations and 9B allow four. Runtime therefore reflects backbone cost, scheduling, simulation, CPU contention, and I/O together. RoboTwin evaluation videos were disabled to reduce rendering and storage overhead, so a complete video archive is unavailable for these runs. LIBERO, LIBERO-plus, and RoboCasa retain video outputs used for aggregation or diagnosis.

Publishing these conditions helps readers distinguish workflow completion time from per-request speed. Future measurements will include policy-latency distributions, environment-step throughput, and execution rates under fixed hardware, inputs, and concurrency.

The 9B Clean and Randomized workflows take 4.51 and 6.38 hours, respectively. Their shorter elapsed times than the Qwen3.5-4B workflows do not establish faster 9B inference: the runs occurred under different software and shared-host conditions. A fixed-concurrency latency experiment is required to make that comparison.

3.2.8 Task-Level Profiles and Uneven Scale Benefits

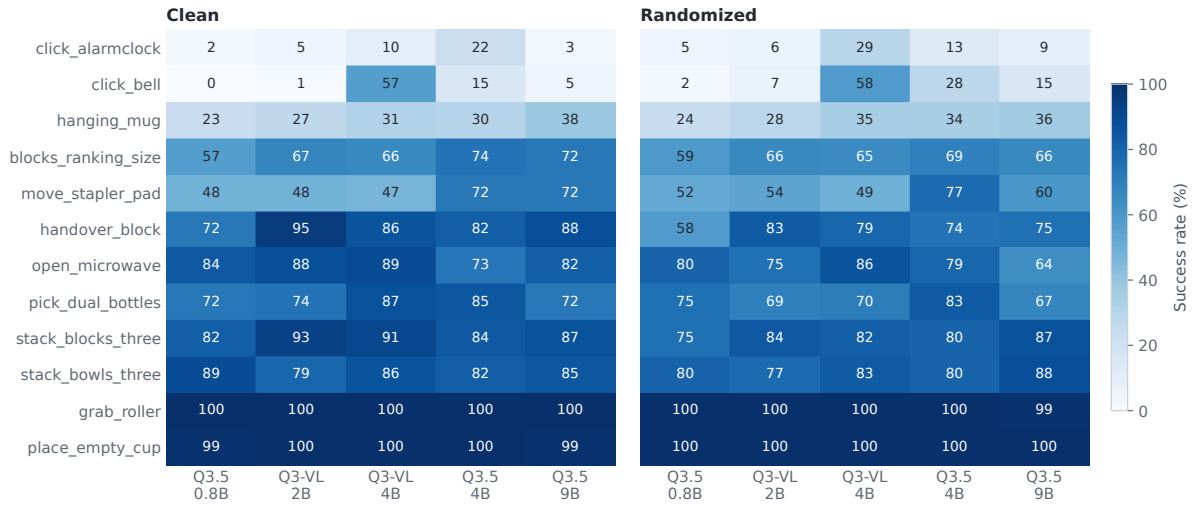


Figure 6. Success rates for selected tasks. The columns correspond to 0.8B, 2B, the two 4B configurations, and 9B. Values are percentages. Precise contact, hanging, handover, and stacking tasks exhibit different responses to backbone scale. All 50 task counts appear in Appendix B.

Scale benefits vary by task. Clean success on `click_bell` increases from 0%/1% for the two small configurations to 57% for Qwen3-VL-4B, while Qwen3.5-4B reaches 15%. This motivates examining interactions

between backbone family and task rather than relying solely on averages. Tasks such as `grab_roller` are nearly saturated across scales, whereas `hanging_mug` remains weak throughout.

Despite similar aggregate 4B scores, `move_stapler_pad`, `open_microwave`, and `click_bell` show substantial differences. These observations motivate diagnosis of data coverage, visual localization, and contact timing; they do not establish a particular module as the cause. Reported anomalies in action-dimension correlations are likewise diagnostic leads rather than proven controller-interface errors.

The 9B results further demonstrate uneven scale benefits. Compared with Qwen3.5-4B, `hanging_mug` improves from 30/34 to 38/36 Clean/Randomized successes and `stack_bowls_three` improves from 82/80 to 85/88. In contrast, `click_alarmclock` falls from 22/13 to 3/9 and `pick_dual_bottles` falls from 85/83 to 72/67. Larger capacity does not consistently remove the contact and manipulation bottlenecks in this recipe. Action-expert capacity, selected VLM feature layers, and data scale are plausible ablation axes, rather than established explanations.

3.2.9 LIBERO and Zero-Shot Robustness

Table 6. ABot-M0 with Qwen3.5-4B on single-arm tasks and perturbations. LIBERO-plus uses the same 40k LIBERO checkpoint without additional training; denominators differ by suite.

	LIBERO		LIBERO-plus	
Suite	Success / total	Rate (%)	Success / total	Rate (%)
Spatial	487/500	97.40	2072/2402	86.26
Object	500/500	100.00	2100/2518	83.40
Goal	488/500	97.60	1953/2591	75.38
Long	460/500	92.00	1902/2519	75.51
Overall micro-average	1935/2000	**96.75**	8027/10030	**80.03**

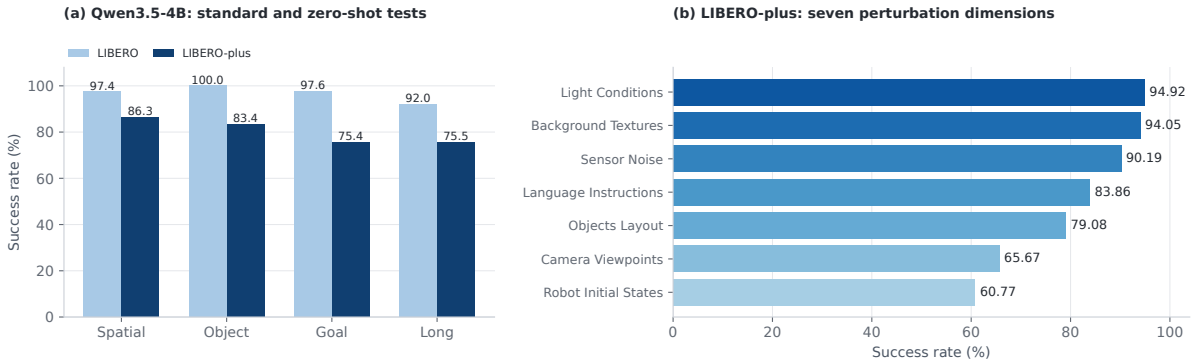


Figure 7. Performance by suite on LIBERO and LIBERO-plus, and by perturbation category on LIBERO-plus. LIBERO uses 50 episodes per task; LIBERO-plus uses one per variant. The two benchmarks are not pooled.

Standard LIBERO success is 96.75%, including 92.00% on the Long suite. Without additional training, the micro-average across all 10,030 LIBERO-plus variants is 80.03%. This exposes perturbation sensitivity beyond performance on the standard tasks.

Table 7. LIBERO-plus perturbation categories, reaggregated from variant-level outcomes and task classifications.

Category	Success / total	Rate (%)
Background Textures	1012/1076	94.05
Camera Viewpoints	1050/1599	65.67
Language Instructions	1289/1537	83.86
Light Conditions	1084/1142	94.92
Objects Layout	1206/1525	79.08
Robot Initial States	942/1550	60.77
Sensor Noise	1444/1601	90.19

Robot initial states and camera viewpoints yield 60.77% and 65.67%, below lighting (94.92%) and background textures (94.05%). The gaps motivate improvements to geometric reasoning and state conditioning. However, categories differ in perturbation difficulty and composition; the comparison does not establish causality or demonstrate understanding of all language variations.

Deduplication and result transparency. Original outputs contain duplicates for 101 tasks, including three tasks with conflicting success labels. We use (suite, task id) as the unique key and retain the output with the latest modification time as the rerun record, obtaining 8,027 successes. Assigning all three conflicting tasks either failure or success bounds the overall score at 80.01–80.04%. The selected records and all alternatives are preserved in the delivered audit data. The rerun-selection rule should be checked against run logs before formal publication. With only one episode per variant, this evaluation does not measure repeat-execution stability for individual variants.

3.2.10 RoboCasa: High-Dimensional Control and Failure Diagnosis

RoboCasa GR1 Tabletop uses 29-dimensional absolute joint targets: seven dimensions per arm, six per hand, and three for the waist. Evaluation covers 24 tasks with 50 episodes each, totaling 1,200 rollouts. Each episode allows up to 720 environment steps, and each prediction executes 12 action steps. Records report approximately 20 hours 44 minutes for 50k training updates on four H200 GPUs, averaging approximately 1.405 seconds per update. This timing is an approximate recorded statistic, unlike the exact RoboTwin wall-time values.

The policy succeeds in 574/1200 episodes (47.83%). Task scores range from 16% to 76%, with lower success on tiered shelves and some placement-and-closure tasks. Transferring an object from a cutting board to a pan reaches 76%.

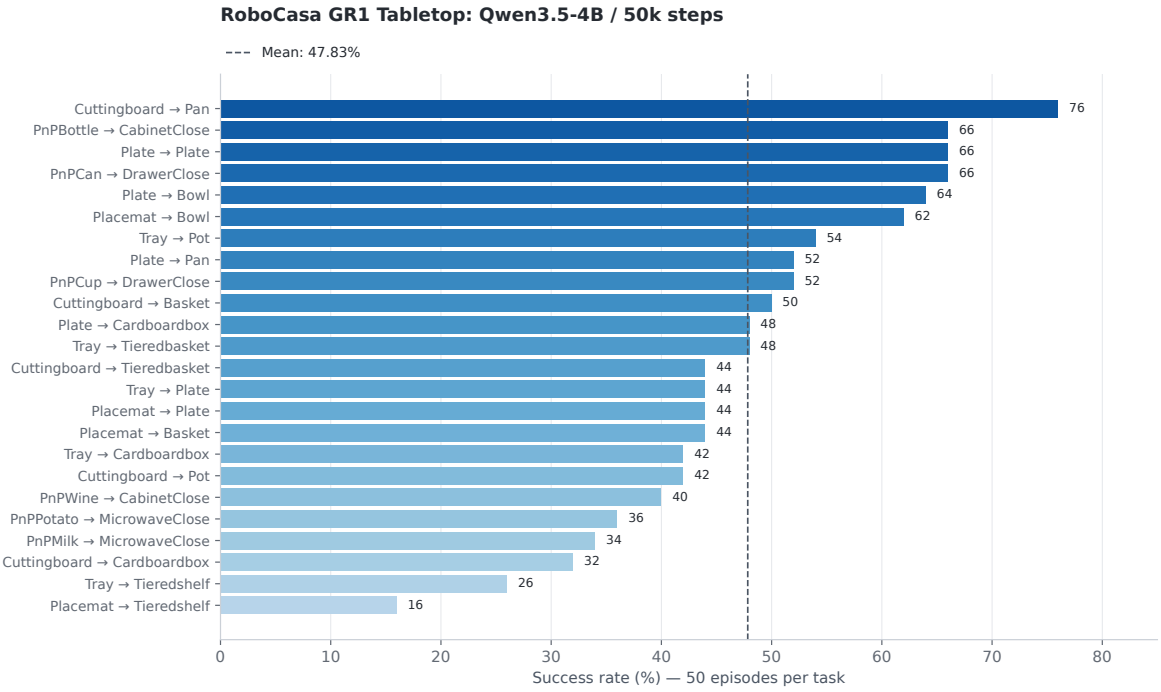


Figure 8. All 24 RoboCasa GR1 Tabletop tasks. The dashed line marks mean success of 47.83%. Equal counts of 50 episodes per task make macro- and micro-averages identical. Figure labels are abbreviated; full task names are retained in the data tables.

Process evidence supports targeted diagnosis. Existing failure videos and probe records show rollouts that fail to approach an object before grasping, sweep it away, or continue until timeout after an unsuccessful grasp. In one tracked sequence, the end effector remains approximately 0.2–0.3 m from the target and the object barely moves, while commanded and measured joint angles differ by approximately 0.01–0.06 rad. For these observed trajectories, the evidence motivates checking the generated policy actions before attributing failure to controller tracking.

The experimenters checked action ordering, normalization statistics, absolute-joint semantics, and state encoding. An early loss plateau and outputs close to current state are consistent with a shortcut that ex-

ploits temporal continuity. **The inertial-shortcut and low-learning-rate explanations remain hypotheses**: probes do not cover every failure, and no controlled learning-rate ablation establishes causality. Follow-up experiments should vary learning rates and state conditions, measure dependence on visual input, and add periodic closed-loop monitoring.

3.2.11 CALVIN: Continuous Language-Conditioned Task Chains

Training and interfaces. ABot-M0 with Qwen3.5-4B is trained independently on demonstrations from CALVIN environments A/B/C/D and evaluated in environment D. The converted LeRobot v2.1 dataset contains 17,870 episodes and 1,071,743 frames, with a 200×200 static camera and an 84×84 wrist camera. Images are resized for the policy interface. This ABCD $\rightarrow$ D setting includes D in training; it does not test transfer to a held-out environment D.

Training uses four H200 GPUs, global batch 64, 34,000 updates, seed 42, frozen VGGT, and a DiT-B action expert. Base/VLM/action learning rates are 2.5×10^{-5}, 10^{-5}, and 10^{-4}, with 5,000 warmup updates and a cosine schedule. The action horizon is eight. CALVIN's seven action channels (relative end-effector translation, rotation, and gripper) occupy the right-arm half of the 14-dimensional internal representation; the configured state layout is 16-dimensional after padding. The saved training script requests `include_state=false`, and the inspected rollout client sends camera images and language without an explicit robot-state field. Configured state dimensions therefore do not establish active proprioceptive conditioning.

The final run completes in 14:39:51 (14.66 hours; 58.7 GPU-hours), with approximately 41.2 nominal samples/s and a 13.27 GB final checkpoint. Its last logged action loss is 0.02949. The console file also contains preliminary runs; only the final uninterrupted 340 observations are used for its loss curve in Appendix C. Preliminary observations and the selection rule are preserved in the audit.

Evaluation protocol. Evaluation uses the stored list of 1,000 five-task sequences and CALVIN's task oracle. The environment resets to the prescribed initial condition at the start of a sequence and carries physical state between successful subtasks; the first failure terminates the chain. Each subtask has a 360-step limit. A single simulation client communicates with a separate policy server. In the inspected client, the executed chunk length is read from the checkpoint configuration and equals eight; the wrapper's `replan_steps=5` field does not control this interval. Policy buffers reset between subtasks. Evaluation wall time is recorded as approximately 10 hours 16 minutes, rather than a fixed-hardware latency measurement.

Table 8. CALVIN ABCD $\rightarrow$ D: successful prefixes for the final 34k Qwen3.5-4B checkpoint. Every row uses the same 1,000 sequences.

Consecutive tasks completed	Sequence count	Rate (%)
≥ 1	889/1000	88.9
≥ 2	798/1000	79.8
≥ 3	721/1000	72.1
≥ 4	646/1000	64.6
≥ 5	571/1000	57.1
Average successful sequence length (maximum 5)		**3.625**

The model completes at least one task in 88.9% of sequences and all five in 57.1%. The five prefix counts sum to 3,625 successes, giving $3,625/1,000 = 3.625$. The task table independently sums to the same 3,625 successes across 4,054 reached subtask attempts. The latter denominator includes the 429 failures that terminate incomplete chains, rather than all 5,000 scheduled task positions. Sequence-level composition and conditional task success must therefore remain distinct.

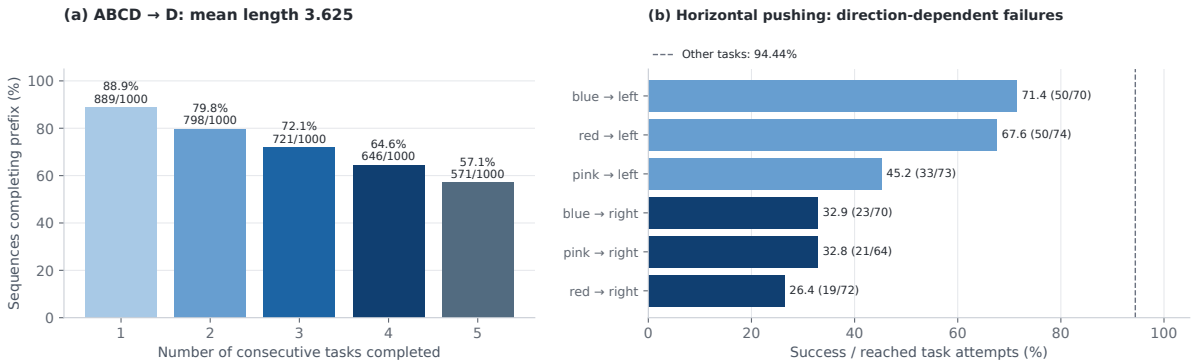


Figure 9. CALVIN chain completion and horizontal pushing. The left panel uses all 1,000 sequences as its denominator. The right panel uses actual reached attempts for each push task; the dashed reference aggregates the other 28 task types. Task frequencies and positions differ, so the reference is diagnostic rather than a matched causal comparison.

Horizontal pushing is the main observed bottleneck. Six left/right block-pushing tasks account for 196/423 successes (46.34%), while the other 28 task types account for 3,429/3,631 (94.44%). Rightward pushes succeed in 63/206 attempts (30.58%), compared with 133/217 (61.29%) for leftward pushes. In total, 26 of the 34 task types reach at least 85% conditional success. `push_into_drawer` is retained among the other tasks because the horizontal block-pushing group is explicitly defined by the six color/direction tasks.

Table 9. CALVIN failure groups. Push left and Push right partition Horizontal push; Other tasks form its complement. Rates are conditional on reaching a task.

Task group	Success / attempts	Rate (%)
Horizontal push	196/423	46.34
Push left	133/217	61.29
Push right	63/206	30.58
Other tasks	3429/3631	94.44

The direction difference motivates checking demonstration counts, signed action distributions, initial object positions, and normalization statistics, followed by failure replay and action-horizon ablations. Contact loss during an eight-step open-loop chunk is another candidate explanation. No controlled experiment yet isolates these effects, and the aggregate rates do not justify a numerical prediction of improvement after fixing pushing.

Result selection and evidence quality. The source JSON contains two occurrences of the same top-level key: an earlier ten-sequence smoke test and the final complete evaluation. We select the final occurrence and preserve both records in `calvin_audit.json`; the smoke test is not pooled. The narrative source table omits `close_drawer`, repeats `lift_pink_block_drawer`, and overstates the number of tasks above 85%. The delivered table follows all 34 JSON task keys. Chain and task totals are internally consistent, but a per-sequence outcome file and a complete failure-video archive are unavailable in the inspected artifacts. The stored sequence list and evaluation code are hashed for provenance; external score comparisons should additionally match evaluator versions and execution settings.

3.2.12 Interpretation and Evidence Limits

The results support discussions of scale trade-offs under the specified ABot-M0 recipe and simulation settings. The following limits govern their interpretation:

- Five RoboTwin configurations have complete scores; Qwen3.5-2B has completed training, but its evaluation remains incomplete in this snapshot.
- Each completed configuration uses one training seed. One hundred episodes per task characterize that checkpoint, not variation across training runs.
- Model scales refer to nominal VLM sizes. Full policies also contain spatial modules, action experts, and projections whose sizes may change with hidden width.
- Update count, batch, and data are controlled, while backbone pre-train histories, dependency versions, and kernel paths remain systematic differences.

- Memory lacks a uniform measurement protocol, and rollout time depends on concurrency and simulation load. Neither establishes minimum deployment memory or a strict latency advantage.
- Action spaces, training steps, and learning rates differ across datasets. The cross-dataset scores do not form a single scaling law. This report includes no real-robot results.
- CALVIN measures successful prefixes of continuous task chains. Its ABCD training includes D; conditional task rates and sequence length do not constitute a held-out-environment transfer score. Available outcomes are aggregate records rather than independently audited per-sequence traces.

These evidence gaps should accompany performance and weights so readers can identify what supports each conclusion and what measurements remain necessary.

4 Conclusion

RoboSeed builds an extensible matrix of backbones, datasets, and action methods for robotics teams with different computing resources, while supporting rapid training and open reproduction of emerging techniques. Under the tested RoboTwin recipe, smaller backbones retain substantial task success with lower training time and GPU-hours. The two 4B backbones have similar aggregate performance but different task profiles and engineering costs; completed 9B evaluation shows that further scaling does not improve the observed success rate. LIBERO-plus and RoboCasa further show that high standard-task success and low training loss do not fully characterize closed-loop capability. CALVIN extends this evidence to continuous task composition: a mean successful length of 3.625 coexists with a clear horizontal-pushing bottleneck and direction asymmetry. These findings favor task-specific diagnosis and controlled changes to the recipe alongside expansion of model scale.

RoboSeed's release goals cover performance, weights, and experimental processes. Memory, convergence, execution conditions, and failure evidence help the community choose models by observed cost, reproduce results under explicit protocols, and investigate capability bottlenecks. Continued expansion of the matrix will accumulate comparable, traceable evidence for practical model use.

5 Future Work

Expand backbones and scales. Complete Qwen3.5-2B evaluation, then add smaller, larger, and additional VLM families. Use the observed 9B result to test changes in action-expert capacity, feature selection, data coverage, and training duration through controlled ablations. Complement fixed-update comparisons with fixed-GPU-hour studies and multiple training seeds to distinguish budget effects from run variation.

Expand datasets and methods. Evaluate more tasks, embodiments, and data scales, and extend replaceable interfaces to additional VLA methods. Preserve checks of dataset semantics, action conventions, and evaluator versions, while reporting adaptation cost and task performance separately.

Diagnose CALVIN composition failures. Audit the six horizontal-pushing task distributions and signed action statistics, retain per-sequence outcomes and representative failure videos, and compare shorter and longer action chunks under the same sequence list. Evaluate intermediate checkpoints and multiple training seeds before estimating gains. Separate ABCD $\rightarrow$ D from held-out-environment settings such as ABC $\rightarrow$ D.

Rapidly evaluate community techniques. Establish a workflow from compatibility checks and short training runs through screening rollouts and complete evaluation. Verify active acceleration kernels when assessing new architectures.

Strengthen process measurements. Standardize memory sampling across allocated, reserved, and device monitoring readings, with explicit units, peak windows, and device capacity. Publish training-stage runtimes, losses, and anomalies. Record policy p50/p95 latency, simulation throughput, success counts, exclusions, and retries under fixed concurrency, alongside representative success and failure videos.

Complete reproducible releases. Package weights with effective configurations, dataset versions, normalization statistics, code and dependency versions, checksums, and task-level outcomes. The project repository at `https://github.com/PrimitiveX/AMseed` will be expanded with experimental records and release materials as coverage grows. Verified download entries will accompany formal weight release. Real-robot experiments will test whether the observed simulation trade-offs carry over to deployment.

A Experimental Details and Evidence Completeness

A.1 Dependencies, Interfaces, and Evaluation Settings

Recorded RoboTwin training dependencies are Python 3.12.3, PyTorch 2.8.0+cu128, accelerate 1.5.2, DeepSpeed ZeRO-2, and bf16 without CPU offload. Qwen3-VL uses transformers 4.57.0, and Qwen3.5 uses 5.15.1. These are run-specific records; reproduction should pin compatible combinations. Effective configuration snapshots saved during training take precedence over repository defaults modified afterward.

Table 10. Key interfaces and evaluation settings. LIBERO-plus entries describe this script snapshot rather than a universal protocol for other implementations.

Setting	RoboTwin	LIBERO / LIBERO-plus	RoboCasa GR1
Training updates	150k	40k / no extra training	50k
Global batch	64	64	64
State / action dimensions	14 / 14	7 / 14 in config; state input to be verified	58 / 29
Training action horizon	50	10	16
Integration steps	4	4	4
Training seed	42	42	42
Evaluation seed	Launch argument 0	Script default 7	Not recorded in summary
Episodes per task	100 per scene	50 / 1 per variant	50
Environment step limit	Task dependent	220 / 280 / 300 / 520 by suite; 10 settling steps	720
Rollout videos	Disabled	Available	Available

The LIBERO-plus script inherits the four suite-specific step limits and settling period. Reproduction and external comparisons should check these settings, executed action chunks, random states, and environment versions. Complete task coverage alone does not imply equivalence to every public evaluation protocol.

A.2 Available Evidence and Remaining Measurements

Table 11. Evidence completeness in this snapshot. Planned artifacts are distinguished from verified measurements.

Evidence	Current status	Follow-up
RoboTwin success	Original results checked against final log counters; 50 tasks per scene	Multiple training seeds and external protocol checks
Configurations and final weights	Five evaluated RoboTwin configurations, one training-only extension, and CALVIN final weights; file sizes measured	Versioned release configurations and weight checksums
Loss convergence	Observations extracted from console logs	Standard exports of raw metrics
Training memory	Approximate recorded mean and maximum readings	Raw traces, units, and capacity checks
Rollout time and concurrency	Workflow times and concurrency limits recorded	Policy latency, task time, and worker activity
LIBERO / RoboCasa counts	Reconstructed video verdicts and task denominators	Independent episode tables and exception logs
LIBERO-plus coverage and duplicates	All variants covered; duplicates and conflicts preserved	Confirm rerun replacement using run IDs and logs
CALVIN chains and task counts	Final aggregate record selected; 34 task types and chain totals checked	Per-sequence traces, evaluator commit, and failure videos
Public repository and weight entries	Project repository provided; release materials under expansion	Weight download links and dependency documentation

A.3 Reproducing the Data and Figures

The delivered `data/` directory contains five completed RoboTwin summary and task tables, six training-loss series, a training-only extension record, task outcomes for LIBERO, LIBERO-plus, and RoboCasa, and LIBERO-plus category statistics and duplicate audits. CALVIN adds prefix counts, all 34 conditional task counts, failure-group statistics, final-run loss observations, the original aggregate JSON, and a duplicate-key selection audit. The `source_manifest.json` file records original paths, configuration mappings, and evidence-file hashes. Original directory aliases appear only in provenance materials; method names in the report remain consistent. Video outcomes are reconstructed from environment-success labels in filenames, rather than independently relabeled through frame-by-frame inspection of every rollout.

The read-only `scripts/collect_data.py` extracts evidence and checks coverage and denominators. The scripts `make_tables.py` and `draw_figures.py` generate manuscript tables and figures from shared data. Figures are exported as PDF, SVG, and PNG; framework diagrams remain editable vectors. All new illustrations use blue, black, white, and gray. Original experiment files and weights are preserved.

B Complete RoboTwin Task Counts

Every numeric column reports successes out of 100 episodes, numerically equal to success rate in percent. C and R denote Clean and Randomized. Q3 denotes Qwen3-VL and Q3.5 denotes Qwen3.5. All five columns of backbone pairs contain complete evaluations; the newly trained Qwen3.5-2B is excluded pending completion of its evaluation.

Table 12. Success counts for all 50 RoboTwin tasks.

Task	Q3.5-0.8B		Q3-2B		Q3-4B		Q3.5-4B		Q3.5-9B	
	C	R	C	R	C	R	C	R	C	R
adjust_bottle	100	100	100	100	100	100	100	99	100	100
beat_block_hammer	83	82	94	88	94	92	89	83	96	87
blocks_ranking_rgb	91	93	93	93	92	94	98	91	94	90
blocks_ranking_size	57	59	67	66	66	65	74	69	72	66
click_alarmclock	2	5	5	6	10	29	22	13	3	9
click_bell	0	2	1	7	57	58	15	28	5	15
dump_bin_bigbin	97	93	97	97	96	97	97	99	96	99
grab_roller	100	100	100	100	100	100	100	100	100	99
handover_block	72	58	95	83	86	79	82	74	88	75
handover_mic	100	99	100	100	98	98	100	99	98	99
hanging_mug	23	24	27	28	31	35	30	34	38	36
lift_pot	99	100	100	100	100	100	100	98	100	100
move_can_pot	83	71	87	83	84	74	82	86	73	75
move_pillbottle_pad	93	86	93	92	96	95	93	90	96	94
move_playingcard_away	97	95	97	98	100	98	99	100	97	99
move_stapler_pad	48	52	48	54	47	49	72	77	72	60
open_laptop	95	98	98	99	97	99	97	100	97	98
open_microwave	84	80	88	75	89	86	73	79	82	64
pick_diverse_bottles	41	59	69	67	71	59	63	61	64	57
pick_dual_bottles	72	75	74	69	87	70	85	83	72	67
place_a2b_left	81	90	88	87	93	96	95	97	93	94
place_a2b_right	78	83	86	83	91	92	91	88	95	94
place_bread_basket	85	81	85	81	87	77	92	86	91	86
place_bread_skillet	89	81	84	83	82	89	94	86	91	88
place_burger_fries	99	98	96	98	95	98	99	95	99	98
place_can_basket	70	63	75	76	79	75	81	72	70	72
place_cans_plasticbox	100	92	100	96	100	99	98	96	99	98
place_container_plate	100	99	99	100	98	99	100	100	99	100
place_dual_shoes	85	82	80	79	82	84	84	85	75	81
place_empty_cup	99	100	100	100	100	100	100	100	99	100
place_fan	94	91	96	98	99	97	94	97	98	94
place_mouse_pad	68	68	75	78	73	70	82	73	76	73
place_object_basket	88	88	92	89	88	86	89	88	90	86
place_object_scale	88	80	93	93	95	90	94	93	92	94
place_object_stand	96	88	90	91	94	86	97	92	97	92
place_phone_stand	77	93	87	98	94	97	91	96	91	99
place_shoe	94	96	96	97	95	100	94	97	93	99
press_stapler	95	94	98	97	93	94	96	94	87	88
put_bottles_dustbin	67	79	79	75	75	80	79	89	74	81
put_object_cabinet	91	91	96	96	87	90	91	92	90	95
rotate_qrcode	90	90	90	93	91	94	91	94	84	91
scan_object	74	83	78	88	89	87	81	81	79	77
shake_bottle	100	100	100	100	100	100	98	100	100	100
shake_bottle_horizontally	100	99	100	100	100	99	100	99	100	100
stack_blocks_three	82	75	93	84	91	82	84	80	87	87
stack_blocks_two	100	97	98	99	96	99	98	93	100	98
stack_bowls_three	89	80	79	77	86	83	82	80	85	88
stack_bowls_two	98	97	96	98	96	98	99	98	99	97
stamp_seal	64	65	68	86	75	86	78	83	74	78
turn_switch	60	71	53	70	60	63	71	69	49	67

C CALVIN Configuration, Training, and Complete Task Counts

Table 13. CALVIN reproduction details from the saved configuration and inspected evaluation interfaces.

Setting	Recorded value
Training / evaluation environments	ABCD / D
Demonstrations / frames	17,870 / 1,071,743; converted LeRobot v2.1
Updates / batch / seed	34,000 / 64 / 42
Frozen encoder	VGGT spatial module
Action expert / integration steps	DiT-B, 16 layers / configured 4
Training / executed action horizon	8 / 8 in the inspected shared client
State / action layout	Configured 16 / 14; active CALVIN action channels: 7
Policy observations	Static and wrist cameras plus language; no explicit robot-state field
Normalization	Saved statistics, min–max action unnormalization, right-arm channel selection
Subtask step limit / stop rule	360 / terminate chain at first failure
Evaluation clients / videos	One simulation client / debug video output disabled by default
Training / recorded evaluation time	14:39:51 / approximately 10:16:00

The saved configuration requests a minimum learning rate of 10^{-6}, whereas the last logged scalar learning rate is 4×10^{-7}. Scheduler and parameter-group conventions must be reconciled before claiming an exact effective rate. The source notes describe acceleration dependencies for this later training environment; a matched kernel ablation has not been performed. These details accompany the result without implying a measured speedup relative to earlier 4B runs.

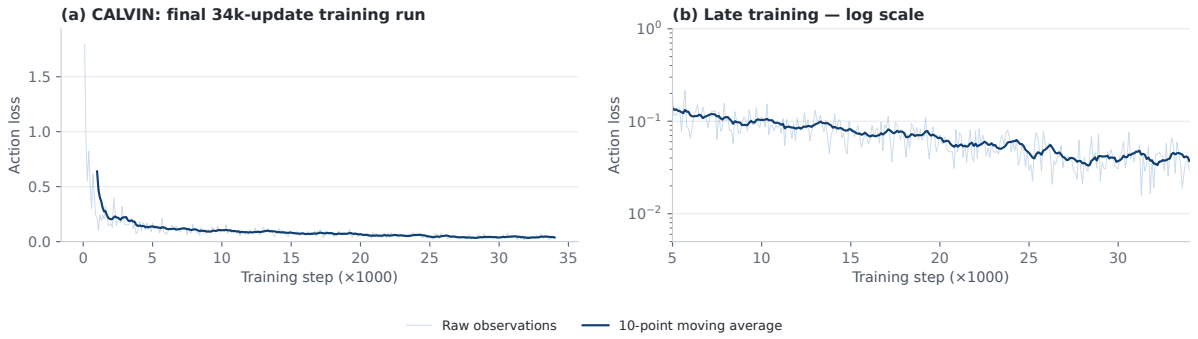


Figure 10. Action loss from the final CALVIN training run. All 340 observations at 100-update intervals are retained; the moving average uses ten observations. Nine earlier observations from preliminary runs in the same console file are excluded from this curve and accounted for in the audit. Final action loss is 0.02949.

Every row below uses the actual number of reached attempts. A task later in a sequence is observed only when its predecessors succeed. The table includes `close_drawer` exactly once and contains 34 unique task types. Totals are 3,625 successes and 4,054 attempts; 26 task types achieve at least 85% conditional success.

Table 14. Complete CALVIN conditional task counts for the selected final evaluation.

Task	Success / attempts	Rate (%)
close_drawer	182/182	100.00
lift_blue_block_drawer	19/19	100.00
lift_blue_block_slider	100/123	81.30
lift_blue_block_table	143/143	100.00
lift_pink_block_drawer	11/11	100.00
lift_pink_block_slider	109/128	85.16
lift_pink_block_table	120/129	93.02
lift_red_block_drawer	17/17	100.00
lift_red_block_slider	106/122	86.89
lift_red_block_table	153/155	98.71
move_slider_left	231/233	99.14
move_slider_right	252/252	100.00
open_drawer	307/320	95.94
place_in_drawer	147/151	97.35
place_in_slider	258/298	86.58
push_blue_block_left	50/70	71.43
push_blue_block_right	23/70	32.86
push_into_drawer	88/102	86.27
push_pink_block_left	33/73	45.21
push_pink_block_right	21/64	32.81
push_red_block_left	50/74	67.57
push_red_block_right	19/72	26.39
rotate_blue_block_left	66/66	100.00
rotate_blue_block_right	70/77	90.91
rotate_pink_block_left	54/55	98.18
rotate_pink_block_right	45/68	66.18
rotate_red_block_left	60/61	98.36
rotate_red_block_right	69/72	95.83
stack_block	153/165	92.73
turn_off_led	138/150	92.00
turn_off_lightbulb	134/135	99.26
turn_on_led	168/168	100.00
turn_on_lightbulb	164/164	100.00
unstack_block	65/65	100.00

References

Yandan Yang et al. ABot-M0: VLA Foundation Model for Robotic Manipulation with Action Manifold Learning. 2026. https://arxiv.org/abs/2602.11236.

Moo Jin Kim et al. OpenVLA: An Open-Source Vision-Language-Action Model. 2024. https://arxiv.org/abs/2406.09246.

Mustafa Shukor et al. SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. 2025. https://arxiv.org/abs/2506.01844.

Tianxing Chen et al. RoboTwin 2.0: A Scalable Data Generator and Benchmark with Strong Domain Randomization for Robust Bimanual Robotic Manipulation. 2025. https://arxiv.org/abs/2506.18088.

Bo Liu et al. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. 2023. https://arxiv.org/abs/2306.03310.

LIBERO-plus. Official implementation: In-depth Robustness Analysis for Vision-Language-Action Models. https://github.com/sylvestf/LIBERO-plus.

RoboCasa. GR1 Tabletop Tasks for GR00T N1. https://github.com/robocasa/robocasa-gr1-tabletop-tasks.

Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. 2022. https://arxiv.org/abs/2112.03227. Official implementation: https://github.com/mees/calvin.